CS2 Week 11

Practical MPC

Exercise Classes

- Shared exercise with Haixiao and Kissan
- Theory recap
- Notes available on Kissan's website: https://kissanv.github.io

& on Haixiao's Polybox: https://polybox.ethz.ch/index.php/s/AReA9YBeJQKxBDn

- Quizzes, Examples & Old exam questions

Kissan's Website

Haixiao's Polybox

Course Schedule

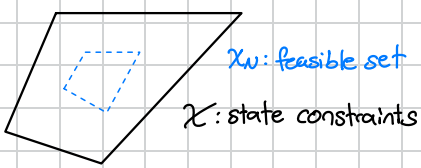
Subject	Week
Intro – Discrete Time Systems	1
Diagonalization, Modal Coordinates, Contr/Obs.	2
State Feedback, Pole Placement, LQR	3
State Estimation, Luenberger Obs., Seperation Principle, LQG	4
DOFC with integral action, Intro to MIMO systems	5
Norms	6
Nonlinear Systems, Lyapunov, Feedback Linearization	7
break	-
Discrete-time Unconstrained Optimal Control	8
Convex Optimization	9
Model Predictive Control	10
Practical Model Predictive Control	11
Robust Model Predictive Control	12
Implementation Methods	13

Last week we focused on mathematical guarantees MPC, we used:

- **Invariant Sets:** We used these to guarantee feasibility. If controller finds safe path now, it will never run out of options in the future.
- **Lyapunov Functions:** We used these to guarantee stability. By making MPC cost function behave like a Lyapunov function → cost always drops, forcing it to origin.

This week, we look at:

- how we can achieve tracking (not forcing state to zero, but track r)
- rejecting disturbances. Constant disturbances cause offset from origin, we want to remove offset, s.t. system converges to desired set point.
- enlargening the feasible set. Constraints restrict set of states, for which opt. problem is feasible. We want it as big as possible.



Tracking

Goal: track given reference r (we care about piecewise constant reference), s.t. $z(k) = Hx(k) \to r \in \mathbb{R}^{n_r}$ as $k \to \infty$

The standard MPC problem regulates system state to origin:

$$U^*(x(k)) = \underset{U_k}{\operatorname{argmin}} \; l_f(x_N) + \sum_{i=0}^{N-1} l(x_{k+i}, u_{k+i})$$

(e.g. quadratic cost: $x^T Q x + u^T R u$)

subj. to $x_k = x(k)$ — measurement

$x_{k+i+1} = Ax_{k+i} + Bu_{k+i}$ — system model

$x_{k+i} \in \mathcal{X}$ — state constraints

$u_{k+i} \in \mathcal{U}$ — input constraints

$U_k = \{u_k, u_{k+1}, \dots, u_{k+N-1}\}$ — opt. variables

⇒ How can we modify this MPC problem to achieve tracking?

We need to change the following:

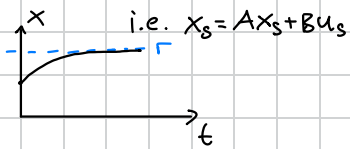
- adapt MPC cost

- adapt terminal set

Steady-State target problem

1. Reference r is achieved by target state x_s if $z_s = Hx_s = r$ (linear comb. of state, maps $x_s \to z_s$)

2. Target state should be a steady-state → there exists an input that keeps system there,



i.e. $x_s = Ax_s + Bu_s$

Summarized:

- $x_s = Ax_s + Bu_s$
- $Hx_s = r$

rewrite in matrix form →

$$\underbrace{\begin{bmatrix} I-A & -B \\ H & 0 \end{bmatrix}}_{(n_x+n_r)\times(n_x+n_u)} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}$$

Further important:

1. If there are any constraints present, (x_s, u_s) has to satisfy them.
2. If the problem has multiple solutions u_s, we compute the 'cheapest' (x_s, u_s) corresponding to r.

translated:

$$\min u_s^T R u_s$$
$$\text{subj. to} \begin{bmatrix} I-A & -B \\ H & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}$$
$$x_s \in \mathcal{X},\ u_s \in \mathcal{U}$$

If on the other hand there is no solution, we compute 'closest' reachable set point to r.

translated:

$$\min (Hx_s - r)^T Q_s (Hx_s - r)$$
$$\text{subj. to } x_s = Ax_s + Bu_s$$
$$x_s \in \mathcal{X},\ u_s \in \mathcal{U}$$

MPC for Reference Tracking

Now that we know, how to calculate (x_s, u_s) to reach r, we want to control the system to this desired steady-state condition, yielding $z(k) \to r$.

So we want to penalize deviations from (x_s, u_s). We use the 2-norm for this.

The MPC is designed as:

Notation: $(u_i - u_s)^T R (u_i - u_s)$

$$\min_U \; \|z_N - Hx_s\|_{P_z}^2 + \sum_{i=0}^{N-1} \|z_i - Hx_s\|_{Q_z}^2 + \|u_i - u_s\|_R^2$$

subj. to [model, constraints]

$$x_0 = x(k)$$

We have now formulated a tracking problem, but we are more familiar with regulation problems. Thus we want to rewrite the problem above to a regulation problem.

- We do this by defining the following deviation variables:

$$\Delta x = x - x_s \qquad \Delta u = u - u_s$$

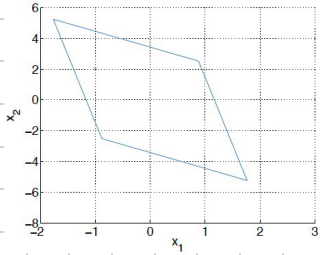
$$\begin{aligned} \Delta x_{k+1} &= x_{k+1} - x_s \\ &= Ax_k + Bu_k - (Ax_s + Bu_s) \\ &= A\Delta x + B\Delta u \end{aligned}$$

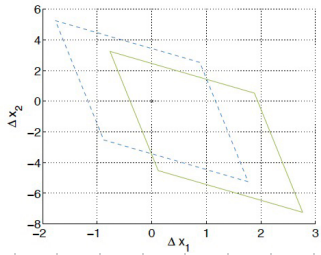
- We also rewrite the constraints for deviation variables:

$$G_x x \le h_x \;\Rightarrow\; G_x \Delta x \le h_x - G_x x_s \quad (-G_x x_s \text{ on both sides})$$

$$G_u u \le h_u \;\Rightarrow\; G_u \Delta u \le h_u - G_u u_s \quad (-G_u u_s \text{ on both sides})$$

Graphically this means a shift:

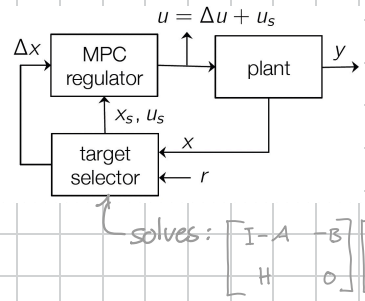




We now have all building blocks we need. We can return to our MPC formulation...

everything put together:

$$\min \sum_{i=0}^{N-1} \Delta x_i^T Q \Delta x_i + \Delta u_i^T R \Delta u_i + V_f(\Delta x_N)$$
$$\text{s.t. } \Delta x_0 = \Delta x(k)$$
$$\Delta x_{i+1} = A\Delta x_i + B\Delta u_i$$
$$G_x \Delta x_i \le h_x - G_x x_s$$
$$G_u \Delta u_i \le h_u - G_u u_s$$
$$\Delta x_N \in \mathcal{X}_f$$



Here we find the optimal sequence of ΔU^* and after apply the input $u_0^* = \Delta u_0^* + u_s$ to the system.

Does this delta formulation preserve the guarantees we proved for regulating to zero?

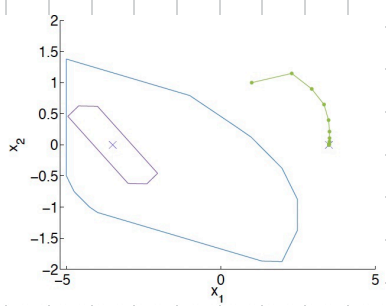
Assume target is feasible with $x_s \in \mathcal{X}$, $u_s \in \mathcal{U}$ and choose terminal constraint $\mathcal{X}_f = \{0\}$ (i.e. $\Delta x_N = 0$, $x_N = x_s$). Then CL system converges to target reference. i.e $x \to x_s$ and therefore $z(k) = Hx(k) \to r$ for $k \to \infty$

Quick remark:

So far we assumed we can measure state and goal is $z(k) = Hx(k) \to r$ for $k \to \infty$
In practice we often can't measure these, but we can measure output $y(k) = Cx(k)$ ← linear combination of states
This is typically handled by running MPC on estimated states.

Summary: MPC for tracking

- Set point tracking problem —(coordinate transformation)→ regulation problem
- If CL system stable ⇝ set point achieved
- Proved stability using same tools as for regulation case
- Difficulties:
  - Terminal set is function of target (x_s, u_s)
  - changing reference can render opt. problem infeasible
- Controller will show offset in case of disturbances or unknown model error



Disturbance Rejection

To eliminate steady-state error we always used integrators up until now.
In constrained control we model the disturbance (estimation).
Then we find control inputs that use this disturbance estimate to remove offset.

Model assuming <u>constant</u> disturbances:

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + B_d d_k \\ d_{k+1} &= d_k \\ y_k &= Cx_k + C_d d_k \end{aligned} \qquad (1)$$

The choice of B_d, C_d is restricted, because the augmented model needs to be observable.

Augmented model is observable iff (A,C) observable and $\begin{bmatrix} A-I & B_d \\ C & C_d \end{bmatrix}$ has full column rank.
i.e. rank $= n_x + n_d \Rightarrow$ max. dimension of disturbance: $n_d \leq n_y$.

can't estimate more disturbances than you have sensors

Why? rewrite (1) at steady-state: $\begin{bmatrix} A-I & B_d \\ C & C_d \end{bmatrix} \begin{bmatrix} x_s \\ d_s \end{bmatrix} = \begin{bmatrix} 0 \\ y_s \end{bmatrix}$

given y_s, d_s must be uniquely defined.

Linear State Estimation

To estimate the state and disturbance we use a Luenberger observer.

State observer for augmented model:

$$\begin{bmatrix} \hat{x}(k+1) \\ \hat{d}(k+1) \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} \hat{x}(k) \\ \hat{d}(k) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u(k) + \begin{bmatrix} L_x \\ L_d \end{bmatrix} \underbrace{(-y(k) + C\hat{x}(k) + C_d\hat{d}(k))}_{\hat{y}(k) - y(k)} \qquad (2)$$

Why does this work?

Look at the error dynamics: (using (1) & (2))

$$\begin{bmatrix} x(k+1)-\hat{x}(k+1) \\ d(k+1)-\hat{d}(k+1) \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x(k)-\hat{x}(k) \\ d(k)-\hat{d}(k) \end{bmatrix} - \begin{bmatrix} L_x \\ L_d \end{bmatrix} \left(C\hat{x}(k) + C_d\hat{d}(k) - Cx(k) - C_d d(k) \right)$$

$$= \left(\begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} + \begin{bmatrix} L_x \\ L_d \end{bmatrix} \begin{bmatrix} C & C_d \end{bmatrix} \right) \begin{bmatrix} x(k)-\hat{x}(k) \\ d(k)-\hat{d}(k) \end{bmatrix}$$

choose $L = \begin{bmatrix} L_x \\ L_d \end{bmatrix}$, s.t. this matrix has stable poles

$\Rightarrow$ error will converge

Now implement the estimated states in (1) at steady-state:

- If $n_y = n_d$
- If observer asy. stable

$$\begin{bmatrix} A-I & B \\ C & 0 \end{bmatrix} \begin{bmatrix} \hat{x}_\infty \\ u_\infty \end{bmatrix} = \begin{bmatrix} -B_d\hat{d}_\infty \\ y_\infty - C_d\hat{d}_\infty \end{bmatrix}$$

This means with measured outputs & input we can reconstruct the state & disturbance.

Offset-free tracking

We can now integrate the above with tracking aswell, i.e. $z(k) = Hx(k) \to r$ for $k \to \infty$

New condition at steady-state:

$$x_s = Ax_s + Bu_s + B_d\hat{d}_\infty \quad \text{(we use our estimate } \hat{d}\text{)}$$
$$z_s = H(Cx_s + C_d\hat{d}_\infty) = r$$

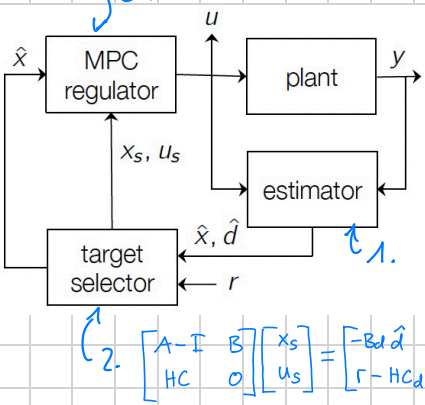
So the adapted target condition which achieves offset-free tracking:

$$\begin{bmatrix} A-I & B \\ HC & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} -B_d\hat{d} \\ r - HC_d\hat{d} \end{bmatrix} \quad (3)$$

Summary: Offset-free tracking

At each sampling time:

1. Estimate $\hat{x}, \hat{d}$
2. obtain (x_s, u_s) from (3) using estimates
3. Solve MPC for tracking



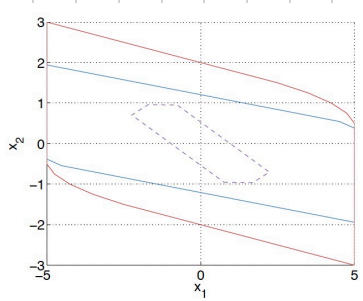
This gives a theoretical result if

- # disturbance states = #outputs
- target steady-state problem is feasible & no constraints active at steady-state

Enlargening Feasible Set

1st approach...

- The terminal constraint reduces feasible set
- Adds state constraints to problems with only input constraints



- Blue line: Feasible set with terminal constraint
- Red line: Feasible set without terminal constraint
- Dashed line: Terminal set

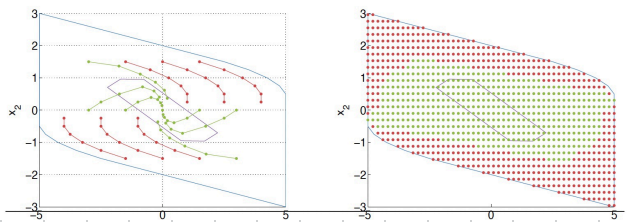
Goal: MPC without terminal constraint with guaranteed stability.

We can remove terminal constraint while maintaining stability if:

- N is large enough
- initial state is within a sufficiently small subset of feasible set (region of attraction)

With this terminal state implicitly satisfies terminal constraint (without being formulated in opt. problem)

$\Rightarrow$ Solution of finite horizon MPC $\hat{=}$ infinite horizon solution

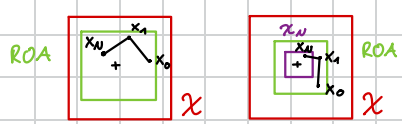


$\Rightarrow$ Controller defined in larger feasible set

$\Rightarrow$ Required horizon length N & region of attraction hard to specify

$\Rightarrow$ Terminal set would 100% guarantee stability, here it doesn't

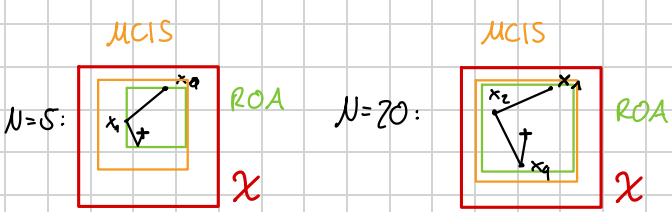
$\Rightarrow$ Region of attraction w/o terminal constraint can be larger than for MPC case



$\Rightarrow$ As N is increased: region of attraction $\longrightarrow$ maximum control invariant set

largest set of starting conditions, where there at least exists one seq. of optimal control inputs

it can look better into the future



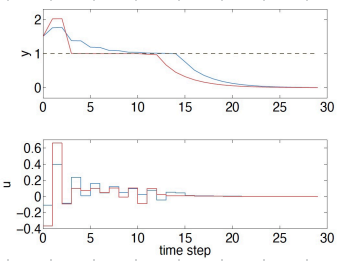
2nd way of enlargenig feasible set...

Soft Constraints

- Input constraints are given by physical constraints on actuators ⇒ hard constraints
- State/Ouput constraints arise from practical constraints on allowed operation range
  ⇒ soft constraints
- Hard constraints complicate the controller ⇒ soften state constraints

Even when the constraints are soft, we are still trying to minimize their violation by

- Minimizing duration of violation
- Minimizing size of violation



Red: Minimum time of violation
Blue: Minimum size of violation

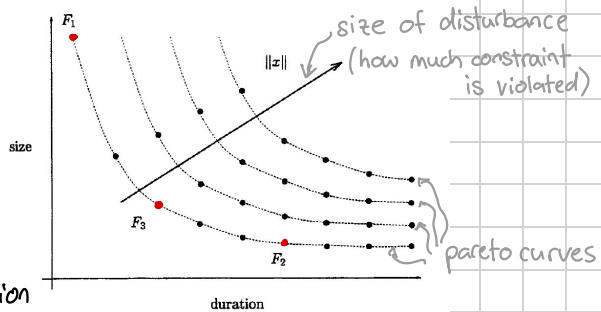
These goal are conflicting though! We can't minimize both simultaneously.

↳ e.g. size of violation ↘ ⇌ duration of violation ↗

To help us find an optimal tradeoff between size & duration of violation we can make use of pareto curves (used offline)

↳ represent tradeoff

- Best operation points lie on pareto optimal curve.
  ⇒ points below cannot be attained
  ⇒ points above are worse



To actually find optimal point on curve engineer is responsible & depends on application

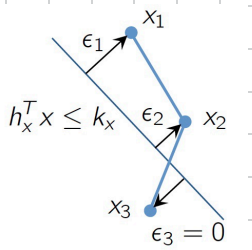
⇒ minimum size? Pick F_2
⇒ minimum duration? Pick F_1

Soft Constraints on MPC

To implement soft state constraints we use slack variables.

$$\min \sum_{i=0}^{N-1} x_i^T Q x_i + u_i^T R u_i + l_\varepsilon(\varepsilon_i) + x_N^T P x_N + l_\varepsilon(\varepsilon_N)$$

$$\text{s.t. } x_{i+1} = Ax_i + Bu_i$$
$$H_x x_i \leq k_x + \varepsilon_i$$
$$H_u u_i \leq k_u$$
$$\varepsilon_i \geq 0$$



- slack variables ε_i to relax state constraints
- penalize amount of constraint violation in cost with l_ε

How do we choose the cost l_ε for soft constraints?

Let's look at the problem more broadly:

Original problem (hard constrained)

$$\min_z f(z)$$
$$\text{subj. to } g(z) \leq 0$$

"Softened" problem

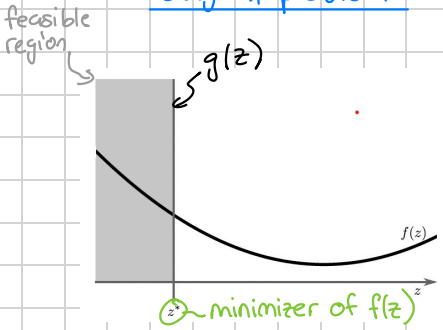
$$\min_{z,\varepsilon} f(z) + l_\varepsilon(\varepsilon)$$
$$\text{subj. to } g(z) \leq \varepsilon$$
$$\varepsilon \geq 0$$

- We only want to interfere when necessary, i.e. when original problem is infeasible
- If the original problem is feasible, the relaxed problem should yield the same result

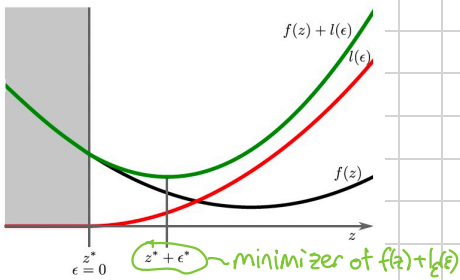
Quadratic Penalty

Let's start by looking at a quadratic cost for the soft constraints.

Original problem



introduce soft constraints →



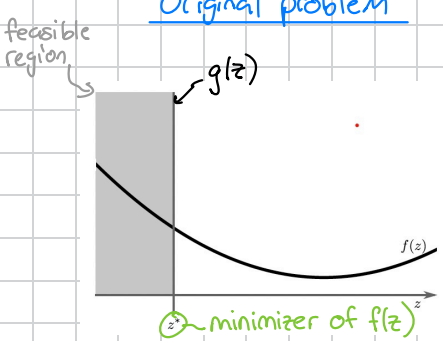
- Constraint: $g(z) = z - z^* \leq 0$
- Quadratic penalty: $l_\varepsilon = s \cdot \varepsilon^2$ for $\varepsilon \geq 0$, 0 else

This is bad! Eventhough problem is feasible, the solution is outside feasible set.

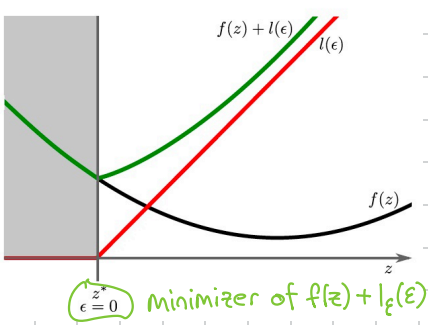
Linear Penalty

What if we try a linear penalty?

Original problem



introduce soft constraints →



- Constraint: $g(z) = z - z^* \leq 0$
- Linear penalty: $l_\varepsilon(\varepsilon) = v \cdot \varepsilon$ for $\varepsilon \geq 0$, 0 else

If we choose v large enough to stop the solver from drifting into infeasible region $\Rightarrow z^*$ minimizer

Why? Because $(f(z)+l_\varepsilon(\varepsilon))' = 0$ and $f'(z) < 0$ so $l_\varepsilon'(\varepsilon) = v \overset{!}{\geq} 0$

In practice one combines linear and quadratic penalty.

- linear: to keep feasible problem inside the constraint
- quadratic: to penalize large violation

If you have multiple constraints we can extend $l_\varepsilon(\varepsilon) = v \cdot \|\varepsilon\|_{1/\infty} + \varepsilon^T S \varepsilon$

Now we know how to choose the penalty, technically can solve MPC problem:

$$\min \sum_{i=0}^{N-1} x_i^T Q x_i + u_i^T R u_i + l_\varepsilon(\varepsilon_i) + x_N^T P x_N + l_\varepsilon(\varepsilon_N)$$

$$\text{s.t. } x_{i+1} = Ax_i + Bu_i$$
$$H_x x_i \leq k_x + \varepsilon_i$$
$$H_x u_i \leq k_u$$
$$\varepsilon_i \geq 0$$

Seperation of Objectives

Often times we want to seperate the problem above into a two step problem

1. Minimize violation over horizon

$$\varepsilon_{min} = \underset{u,\varepsilon}{\operatorname{argmin}} \sum_{i=0}^{N-1} \varepsilon_i^T S \varepsilon_i + v^T \varepsilon_i$$

$$\text{s.t. } x_{i+1} = Ax_i + Bu_i$$
$$H_x x_i \leq k_x + \varepsilon_i$$
$$H_u u_i \leq k_u$$
$$\varepsilon_i \geq 0$$

2. Optimize controller performance

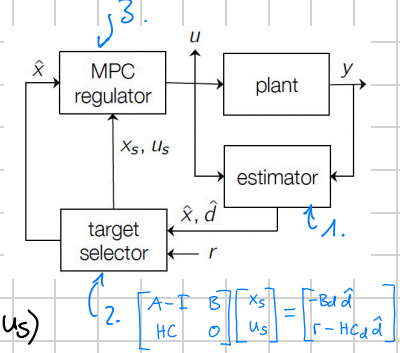
$$\min \sum_{i=0}^{N-1} x_i^T Q x_i + u_i^T R u_i + x_N^T P x_N$$

$$\text{s.t. } x_{i+1} = Ax_i + Bu_i$$
$$H_x x_i \leq k_x + \varepsilon_i^{min}$$
$$H_u u_i \leq k_u$$

⇒ Simplifies tuning, constraints satisfied if possible

⇒ Requires solution of two opt. problems

Summary



Tracking:

- Rewrite problem in delta formulation
- Setup target steady state problem to calculate (x_s, u_s)

$\Rightarrow$ tracking $\hat{=}$ regulation

Disturbance Rejection:

- Augment model including disturbance
- estimate $\hat{x}$ & $\hat{d}$
- adapt target steady state problem using $\hat{d}$

Soft Constraints:

- Recover feasibility when constraints cannot be satisfied
- Introduce slack variables & choose penalty on these
- Violation durations vs size

Exercise

reminder:

tracking steady-state: $\begin{bmatrix} I-A & -B \\ H & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}$

LQR: $u = Kx$, $K = -(R+B^TPB)^{-1}B^TPA$

DARE: $P = Q + A^TPA - A^TPB(R+B^TPB)^{-1}B^TPA$

Consider the system

$$x(k+1) = \begin{bmatrix} 1.1 & 0.1 \\ 0 & 1 \end{bmatrix} x(k) + \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} u(k) \quad (1)$$

$$y(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} x(k) \quad (2)$$

with initial condition $x(0) = [-0.7, 1.4]^\top$ and stage cost $l(x,u) = x^\top Q x + u^\top R u$ with $Q = I_2$, $R = 1$. Design an LQR controller such that $y(k) \to r$ for $k \to \infty$ and $r \in \mathbb{R}$.

1. find (x_s, u_s)

$$\begin{bmatrix} I-A & B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}$$

$$\begin{bmatrix} -0.1 & -0.1 & 0 \\ 0 & 0 & 0.1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_{s,1} \\ x_{s,2} \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ r \end{bmatrix}$$

↳ $x_{s,1} = r$, $u_s = 0$, $x_{s,1} = -r$ ⇝ steady-state: $x_s = \begin{bmatrix} r \\ -r \end{bmatrix}$, $u_s = 0$

2. obtain tracking cost (with delta formulation)

$\Delta x = x - x_s$, $\Delta u = u - u_s$

$$\sum_{i=0}^{\infty} \Delta x_i^T Q \Delta x_i + \Delta u_i^T R \Delta u_i$$

subj. to: $\Delta x_{i+1} = A \Delta x_i + B \Delta u_i$

3. LQR for delta formulation $\Delta u_i = K \Delta x_i$

solve for P: $P = Q + A^TPA - A^TPB(R+B^TPB)^{-1}B^TPA$

LQR: $K = -(R+B^TPB)^{-1}B^TPA$

Input: $\Delta u_i = u - u_s \to u = u_s + \Delta u_i = u_s + K(x - x_s)$